



Université IBN ZOHR

Faculté des Sciences Agadir

Centre d'excellence

Rapport de Projet entrepreneuriat :
Application intelligente pour
l'optimisation agricole basée sur
l'IoT et le Data Analytics

Encadré par :

- Pr.Abdessamad AMAL

Réalisé par :

MAAROUF Yassine IISE G2

HAMDANI Hasna IL G2

Baiha Fatima Ezzahrae ADIA/G1

Année universitaire 2025-2026

Dédicace

Nous dédions ce travail avec une profonde émotion et une immense reconnaissance à nos chers parents,

Qui ont toujours été notre plus grand soutien et notre plus belle source de motivation.

Par leur amour inconditionnel, leur patience infinie et leurs sacrifices silencieux, ils nous ont accompagnés à chaque étape de notre parcours, même dans les moments les plus difficiles.

Aucun mot ne saurait exprimer pleinement notre gratitude pour tout ce qu'ils ont fait et continuent de faire pour nous.

Nous adressons également nos sincères remerciements à nos formateurs,

Qui, par leur encadrement, leur disponibilité et leur engagement, nous ont transmis bien plus que des connaissances.

Leur soutien et leurs conseils ont été essentiels à notre formation et à l'aboutissement de ce projet.

Enfin, nous tenons à remercier toutes les personnes qui nous ont soutenus de près ou de loin,

Par un mot, un conseil ou un encouragement, et qui ont contribué, chacun à sa manière, à la réalisation de notre Projet.

Remerciement

Avant tout, nous remercions Dieu Tout-Puissant de nous avoir accordé la force, la patience et la persévérance nécessaires pour mener à bien ce projet.

Nous exprimons notre profonde gratitude à l'ensemble de nos enseignants et formateurs,

Pour la qualité de l'enseignement dispensé, leur engagement et les connaissances qu'ils nous ont transmises tout au long de notre parcours académique, et qui ont constitué une base essentielle pour la réalisation de ce travail.

Nos sincères remerciements vont également à nos chers parents,

Pour leur soutien inconditionnel, leurs sacrifices, leur patience et leurs encouragements constants.

Leur présence et leur confiance ont été une source permanente de motivation et de réussite.

Nous remercions toutes les personnes qui, de près ou de loin,

Ont contribué à la réalisation de ce projet, par leurs conseils, leur aide ou leur soutien moral.

Enfin, nous adressons nos remerciements à tous ceux qui ont partagé avec nous cette expérience académique et humaine, et qui ont participé, chacun à sa manière, à l'aboutissement de ce travail

Introduction Générale

L'agriculture constitue un pilier fondamental de l'économie marocaine, jouant un rôle essentiel dans la sécurité alimentaire et le développement socio-économique. Cependant, ce secteur fait face à plusieurs défis, notamment le manque de transparence des prix, la dépendance aux conditions climatiques et la faible digitalisation des pratiques agricoles.

Dans ce contexte, l'intégration des nouvelles technologies, telles que l'Internet des objets (IoT) et l'analyse des données (Data Analytics), représente une opportunité majeure pour moderniser le secteur agricole et améliorer la prise de décision des agriculteurs.

C'est dans cette optique que s'inscrit notre projet, qui consiste à développer une application mobile intelligente permettant aux agriculteurs de consulter les prix des produits agricoles, de rechercher des solutions adaptées basées sur l'analyse de l'état du sol.

Cette application combine un système embarqué pour la collecte des données (température, humidité, etc.), une analyse intelligente de ces données, et une interface mobile simple et intuitive pour faciliter son utilisation.

L'objectif principal de ce projet est d'aider les agriculteurs à optimiser leurs rendements, à prendre de meilleures décisions et à adopter une agriculture plus moderne, durable et efficace.

Chapitre 01 : Etude de ***marché***

1. Introduction :

Dans ce chapitre, nous présentons une étude du marché agricole au Maroc afin de mieux comprendre son fonctionnement, ses enjeux et ses opportunités. Cette analyse permet d'identifier les besoins des acteurs du secteur, les problèmes existants ainsi que les solutions pouvant être apportées par notre projet.

L'objectif est de justifier la pertinence de notre application en mettant en évidence les limites du marché actuel et le potentiel de digitalisation du secteur agricole.

2. Étude de Marché (Agriculture au Maroc) :

A. Importance du secteur agricole :

L'agriculture joue un rôle essentiel dans l'économie marocaine. Elle représente environ **10% à 12% du PIB** et emploie près de **30% de la population active**. C'est donc un **secteur stratégique**, à la fois pour la sécurité alimentaire et pour le développement économique du pays.

B. État du marché agricole :

Le marché agricole est dynamique mais reste instable. Il est caractérisé par :

- Une **forte variation des prix** selon les saisons et les régions
- Une **dépendance aux conditions climatiques** (sécheresse, pluie, etc.)
- Un **manque de transparence** entre producteurs, intermédiaires et marchés

C. Problèmes identifiés :

Le secteur agricole fait face à plusieurs défis majeurs :

- Manque de transparence sur les prix
- Faible digitalisation du secteur
- Mauvaise gestion des cultures et des ressources

D. Clients cibles :

Notre solution s'adresse principalement à :

- Agriculteurs
- Coopératives agricoles
- Grossistes et distributeurs.

E. Concurrence :

Les principaux concurrents sont **indirects**, tels que :

- Les souks traditionnels
- Les intermédiaires (middlemen)
- Les réseaux sociaux (Facebook, WhatsApp groups)

Cependant, ces solutions présentent des limites :

- Manque de fiabilité
- Absence d'analyse des données
- Aucune intelligence ou optimisation des décisions

F. Modèle économique :

Le projet peut générer des revenus à travers plusieurs sources :

- **Publicité** (produits agricoles et services)
- **Vente de données agricoles anonymisées** pour analyses de marché

3. Conclusion :

À travers cette étude, nous constatons que le secteur agricole marocain présente des défis importants, tels que le manque de transparence et la faible digitalisation.

Ces constats confirment l'intérêt de développer une solution digitale capable d'améliorer la gestion des activités agricoles, d'optimiser les décisions et de faciliter l'accès à l'information pour les agriculteurs.

Chapitre 02 : Partie
système embarqué
(IoT)

1. Introduction

Ce document présente l'architecture technique du système intégré combinant un microcontrôleur ESP32, un module d'apprentissage automatique (Machine Learning) et une interface web. L'objectif est de mesurer en temps réel les paramètres physico-chimiques du sol (humidité, température, pH, azote, phosphore, potassium), de les transmettre à un modèle prédictif, et d'afficher à l'utilisateur les recommandations de cultures adaptées via une application web.

Le système est développé en deux modes complémentaires : (1) un prototype matériel réel à base de capteurs agricoles dédiés, et (2) une simulation Wokwi permettant de valider la logique embarquée sans dépendance physique aux capteurs.

2. Composants matériels réels

Le prototype physique du système repose sur quatre composants principaux : un microcontrôleur ESP32, un capteur de température et d'humidité de l'air DHT22, un capteur de sol multi-paramètres (NPK), et un capteur de pH analogique. Ces composants ont été sélectionnés pour leur fiabilité, leur disponibilité et leur compatibilité avec les protocoles standards (analogique, I²C, OneWire).

2.1 Microcontrôleur ESP32

L'ESP32 est un microcontrôleur 32 bits dual-core fonctionnant à 240 MHz, intégrant nativement le Wi-Fi et le Bluetooth. Il dispose de 18 entrées analogiques (ADC 12 bits) et de nombreuses GPIO numériques, ce qui le rend particulièrement adapté à la lecture simultanée de plusieurs capteurs. Sa connectivité sans fil permet, dans une version finale, l'envoi direct des données vers le serveur d'inférence ML.



Figure 1 — Carte de développement ESP32 (Wi-Fi + Bluetooth)

2.2 Capteur DHT22 (température et humidité)

Le DHT22 (AM2302) est un capteur numérique mesurant la température (-40 à +80 °C, $\pm 0,5$ °C) et l'humidité relative de l'air (0 à 100 %, ± 2 %). Il communique via un protocole OneWire propriétaire sur une seule broche de données. Dans notre système, il fournit les paramètres atmosphériques contextuels associés aux mesures du sol.

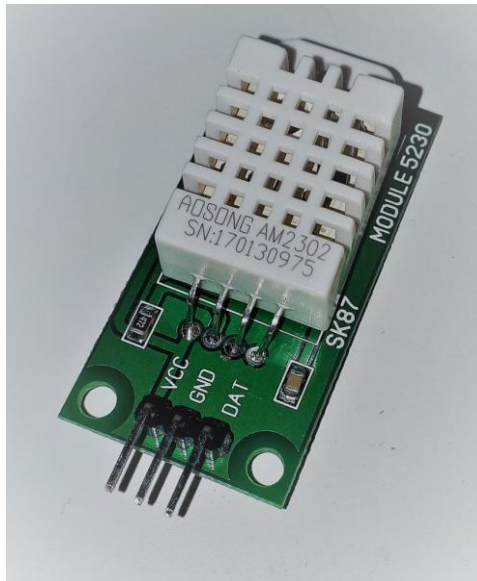


Figure 2 — Capteur DHT22 (AOSONG AM2302) — module 3 broches : VCC, GND, DATA

2.3 Capteur de sol multi-paramètres (NPK)

Le capteur de sol industriel utilisé est une sonde à pointes inox mesurant l'humidité du sol, la température, la conductivité électrique et le pH. Il dispose de plusieurs modes de sortie (0–5 V, 0–10 V, 4–20 mA, RS485). Couplé à un module dédié, il fournit également les concentrations en azote (N), phosphore (P) et potassium (K), paramètres essentiels à la prédiction des cultures adaptées.



Figure 3 — Capteur de sol industriel (Humidité, Température, Conductivité, pH)

2.4 Capteur de pH analogique

Pour des mesures de pH dédiées et calibrables, un capteur de pH analogique avec sonde en verre est utilisé. Le module électronique associé fournit une tension proportionnelle au pH mesuré (0–14), directement lisible par l'ADC de l'ESP32. Deux potentiomètres permettent l'étalonnage (offset et gain).



Figure 4 — Module de pH analogique avec sonde en verre

2.5 Récapitulatif des composants

Composant	Rôle	Interface
ESP32	Microcontrôleur principal — acquisition, traitement, communication	Wi-Fi / USB Série
DHT22	Mesure température et humidité de l'air	OneWire (1 broche)
Capteur NPK	Mesure azote, phosphore, potassium du sol	Analogique / RS485
Sonde pH	Mesure du pH du sol	Analogique (0-5 V)
Écran OLED 0.96"	Affichage local des lectures et de l'état du système	I ² C (SDA/SCL)

3. Simulation sous Wokwi

La plateforme Wokwi permet de simuler intégralement un circuit ESP32 directement dans Visual Studio Code, sans matériel physique. Cela autorise le développement, le débogage et la démonstration de la logique embarquée avant l'assemblage du prototype réel.

3.1 Substitution des capteurs en simulation

Wokwi ne propose pas de modèles natifs pour les capteurs agricoles spécialisés (NPK, sonde pH). Pour reproduire fidèlement leur comportement, ces capteurs sont remplacés par des potentiomètres :

- Le capteur de pH est remplacé par un potentiomètre dont la tension de sortie (0–3,3 V) est convertie en valeur de pH (0–14) par mise à l'échelle logicielle.
- Les concentrations N, P, K sont chacune simulées par un potentiomètre indépendant, dont la tension est convertie en concentration (0–100).
- Le DHT22 est utilisé tel quel — Wokwi propose un modèle natif fidèle.
- Un bouton-poussoir simule l'événement de validation manuelle (envoi vers le module ML).

Cette substitution préserve totalement la logique de lecture, de stabilisation et de transmission : le firmware ESP32 reste identique entre la simulation et le déploiement réel — seul le câblage physique change.

3.2 Schéma global du circuit simulé

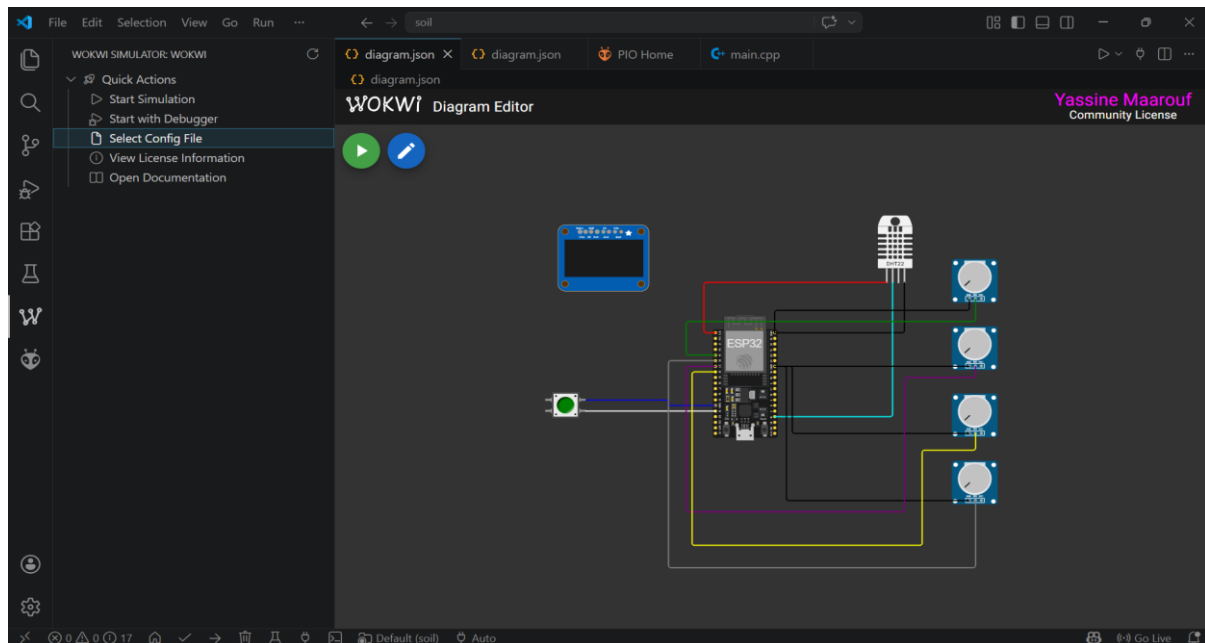


Figure 5 — Vue d'ensemble du circuit Wokwi : ESP32, OLED, DHT22, 4 potentiomètres (pH/N/P/K), bouton

3.3 Détail du câblage

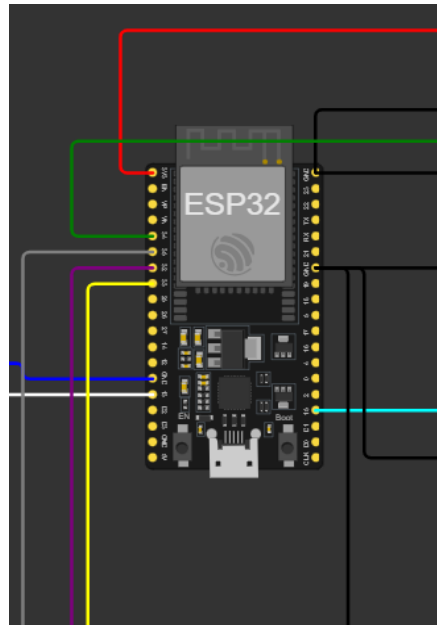


Figure 6 — Câblage central de l'ESP32 (alimentation, bus I²C, entrées analogiques)

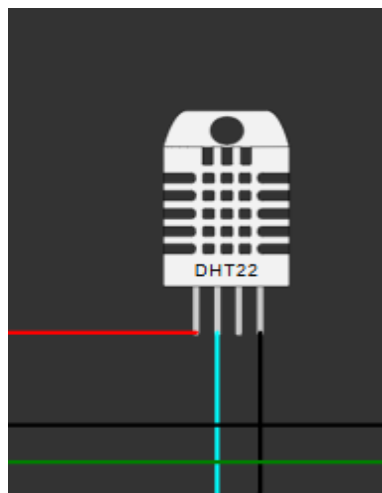


Figure 7 — Capteur DHT22 (broche DATA reliée au GPIO 15)

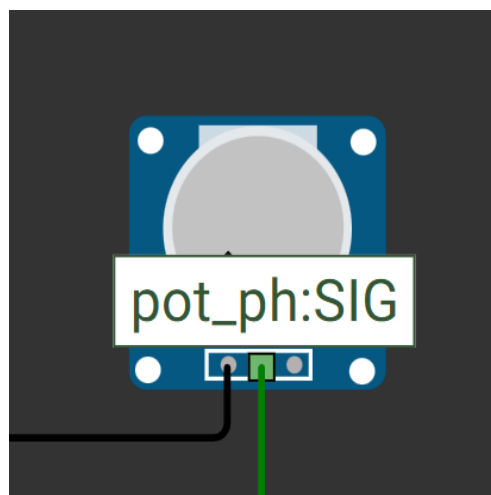


Figure 8 — Potentiomètre simulant le capteur de pH (signal SIG → GPIO 34)

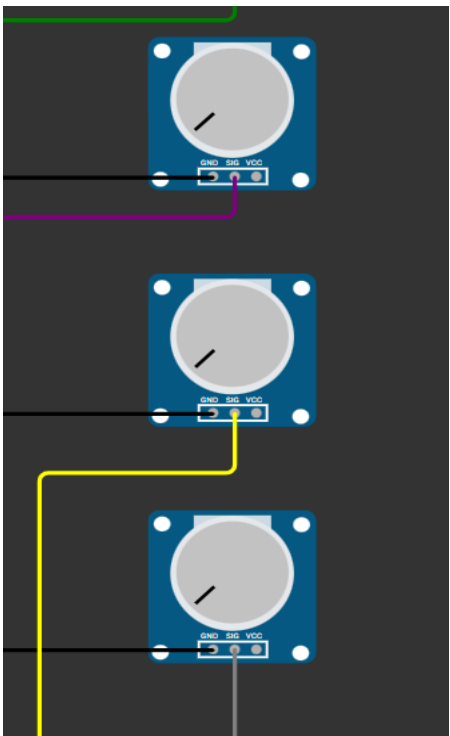


Figure 9 — Trois potentiomètres simulant les concentrations N, P, K (GPIO 32, 33, 35)

3.4 Affectation des broches

Broche ESP32	Composant	Rôle
GPIO 15	DHT22 (DATA)	Lecture température et humidité de l'air
GPIO 34 (ADC)	Potentiomètre pH	Mesure du pH (0–14)
GPIO 32 (ADC)	Potentiomètre N	Concentration azote (0–100)
GPIO 33 (ADC)	Potentiomètre P	Concentration phosphore (0–100)
GPIO 35 (ADC)	Potentiomètre K	Concentration potassium (0–100)
GPIO 13	Bouton-poussoir	Validation manuelle de l'envoi vers le ML
SDA / SCL	Écran OLED SSD1306	Affichage local (I ² C, adresse 0x3C)

4. Code embarqué (firmware ESP32)

Le firmware est développé en C++ (Arduino framework) et compilé via PlatformIO. Il assure quatre fonctions principales : (1) la lecture cyclique des capteurs, (2) la détection de stabilité des mesures, (3) la sauvegarde locale au format CSV, et (4) la transmission JSON vers le module ML sur appui du bouton.

4.1 Définitions et déclarations

```
#define DHT_PIN 15
#define DHT_TYPE DHT22
#define PIN_PH 34
#define PIN_N 32
#define PIN_P 33
#define PIN_K 35
#define BUTTON_PIN 13

#define STABILITY_THRESHOLD 2.0
#define STABILITY_COUNT 3

DHT dht(DHT_PIN, DHT_TYPE);
Adafruit_SSD1306 display(128, 64, &Wire, -1);
```

4.2 Lecture des capteurs analogiques

La fonction readAnalog() convertit la lecture brute de l'ADC 12 bits (0–4095) en valeur physique mise à l'échelle (pH 0–14 ou concentration 0–100).

```
float readAnalog(int pin, float maxVal) {
    return (analogRead(pin) / 4095.0) * maxVal;
}
```

4.3 Détection de stabilité

Pour éviter d'envoyer des mesures bruitées ou transitoires, le firmware vérifie que toutes les valeurs sont stables (variation inférieure à un seuil) pendant trois cycles consécutifs avant de valider l'enregistrement.

```
bool isStable() {
    return (abs(humidity - prevH) < STABILITY_THRESHOLD &&
            abs(ph - prevPH) < STABILITY_THRESHOLD &&
            abs(nitrogen - prevN) < STABILITY_THRESHOLD &&
            abs(phosphorus - prevP) < STABILITY_THRESHOLD &&
            abs(potassium - prevK) < STABILITY_THRESHOLD);
}
```

4.4 Transmission JSON vers le module ML

Lors de l'appui sur le bouton (et si les mesures sont stables), le firmware émet un objet JSON sur le port série, capturé par l'application web qui le relaie au backend Flask.

```
void sendToML() {
    Serial.println("--- SENDING TO ML ---");
    Serial.print("{\"humidity\":"); Serial.print(humidity, 1);
    Serial.print(", \"temperature\":"); Serial.print(temperature, 1);
    Serial.print(", \"pH\":"); Serial.print(ph, 2);
    Serial.print(", \"N\":"); Serial.print(nitrogen, 1);
    Serial.print(", \"P\":"); Serial.print(phosphorus, 1);
    Serial.print(", \"K\":"); Serial.print(potassium, 1);
    Serial.println("}");
}
```

4.5 Boucle principale

```
void loop() {
    humidity    = dht.readHumidity();
    temperature = dht.readTemperature();
    ph          = readAnalog(PIN_PH, 14.0);
    nitrogen    = readAnalog(PIN_N, 100.0);
    phosphorus  = readAnalog(PIN_P, 100.0);
    potassium   = readAnalog(PIN_K, 100.0);

    if (isStable()) stableCount++;
    else { stableCount = 0; dataSaved = false; }

    bool stable = stableCount >= STABILITY_COUNT;

    if (stable && !dataSaved) { saveCSV(); dataSaved = true; }
    if (digitalRead(BUTTON_PIN) == LOW && stable) { sendToML(); delay(500); }

    updateOLED(stable);
    prevH = humidity; prevPH = ph;
    prevN = nitrogen; prevP = phosphorus; prevK = potassium;
    delay(1000);
}
```

5. Résultats de la simulation

La simulation Wokwi a été exécutée avec succès. Le firmware compile sans erreur, les capteurs sont lus à la fréquence de 1 Hz, l'écran OLED affiche les valeurs en temps réel, et la sortie série produit le format CSV puis le JSON attendu lors de l'appui sur le bouton.

5.1 Exécution du projet sous Wokwi

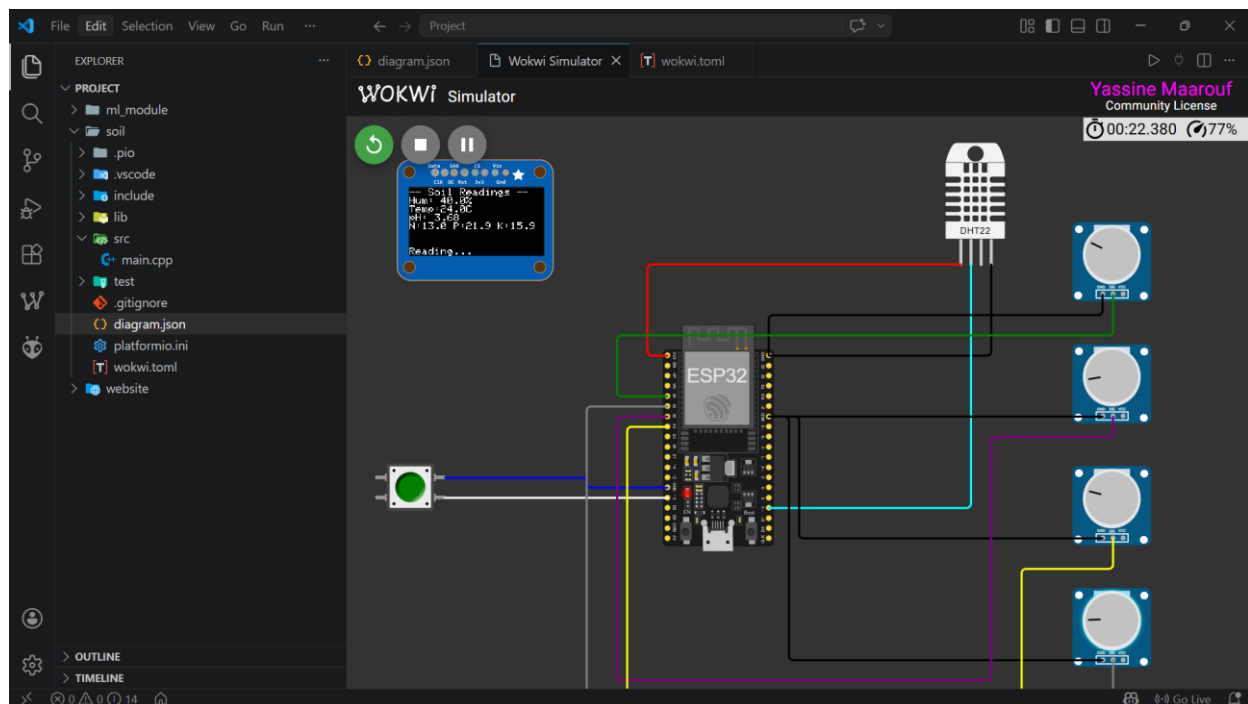


Figure 10 — Projet en cours d'exécution dans Wokwi : OLED actif, simulation à 77 % de vitesse réelle

5.2 Affichage local sur écran OLED

L'écran OLED 128×64 pixels affiche en permanence les six paramètres mesurés ainsi que l'état du système (« STABLE - Press btn » ou « Reading... »). Cela offre un retour visuel immédiat à l'opérateur sur le terrain.

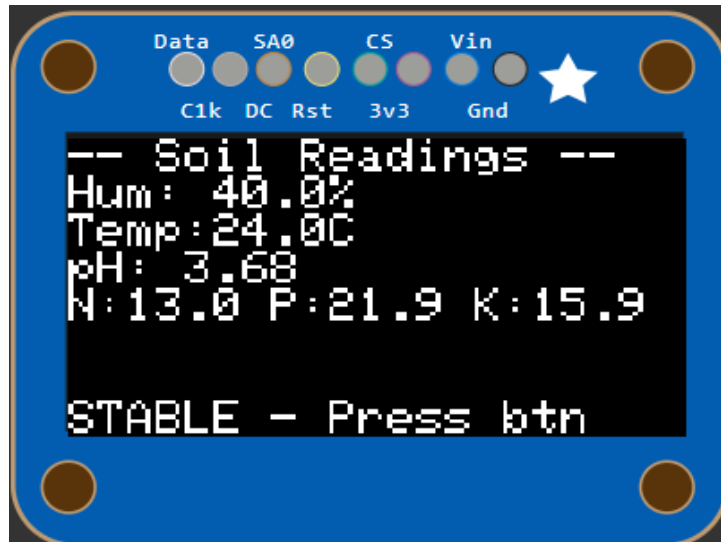


Figure 11 — OLED affichant des lectures stables : Hum 40,0 %, Temp 24,0 °C, pH 3,68, N 13,0, P 21,9, K 15,9

5.3 Sortie série (CSV et JSON)

Le terminal Wokwi confirme la production des deux formats de sortie : un en-tête CSV suivi des valeurs sauvegardées automatiquement à chaque stabilisation, puis le JSON émis sur appui du bouton et destiné à être consommé par l'API Flask du module ML.

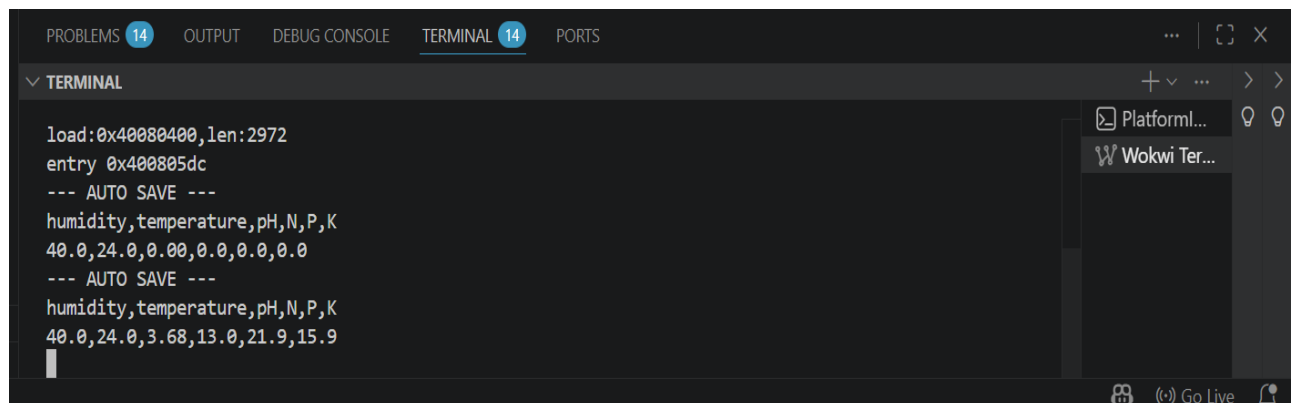


Figure 12 — Terminal Wokwi : sauvegarde CSV automatique avec en-tête humidity,temperature,pH,N,P,K

6. Architecture globale ESP32 → ML → Web

Le système suit une chaîne de traitement linéaire en trois étapes : acquisition embarquée (ESP32), inférence côté serveur (module ML Flask), et restitution utilisateur (application web React). Les trois composants communiquent via des formats standards (Série/JSON et HTTP/JSON).

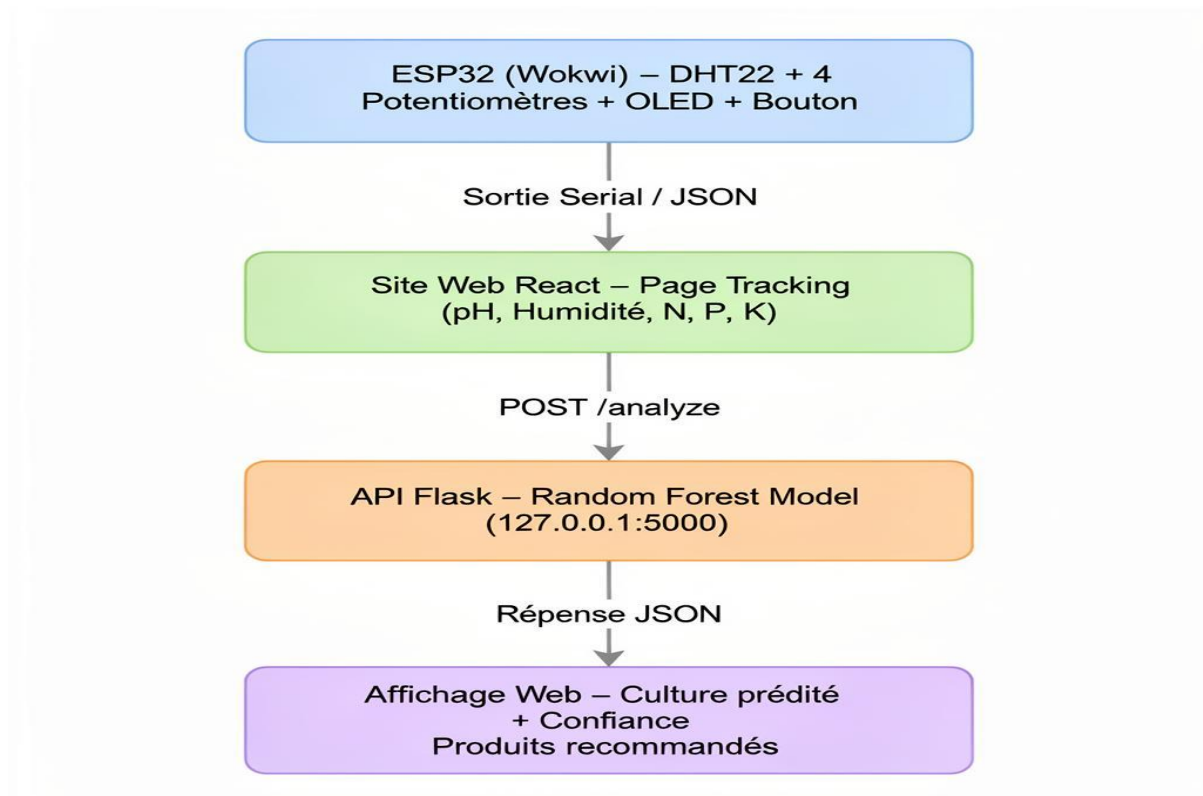


Figure 13 — Architecture globale du système : flux de données de l'ESP32 vers l'utilisateur final

6.1 Étape 1 — Acquisition (ESP32 / Wokwi)

- Lecture cyclique des six paramètres du sol (1 Hz).
- Détection de stabilité sur 3 cycles successifs (seuil $\pm 2,0$).
- Sauvegarde locale au format CSV à chaque stabilisation.
- Émission JSON sur le port série déclenchée par le bouton-poussoir.

6.2 Étape 2 — Inférence (Module ML / Flask)

Le module ML expose une API REST développée avec Flask. L'endpoint POST /analyze reçoit un objet JSON contenant les six paramètres, applique un modèle Random Forest entraîné sur 20 000 échantillons (jeu de données crop-recommandation), et retourne la culture recommandée ainsi qu'un score de confiance et des cultures alternatives.

POST /analyze
Content-Type: application/json

```
{
  "humidity": 40.0, "temperature": 24.0, "pH": 3.68,
  "N": 13.0, "P": 21.9, "K": 15.9
}
```

→ Réponse :

```
{
  "recommended_crop": "rice",
  "confidence": 0.87,
  "alternatives": ["maize", "cotton"]
}
```

6.3 Étape 3 — Restitution (Site Web / React)

L'application web, développée avec React et Vite, propose une page « Tracking » où l'utilisateur saisit (ou reçoit automatiquement) les valeurs lues sur le port série de l'ESP32. La page interroge l'API Flask et affiche la recommandation, le niveau de confiance, et les cultures alternatives sous forme graphique.

6.4 Mode démonstration vs déploiement Wi-Fi

Mode démonstration (actuel)	Mode déploiement (cible)
ESP32 → port série USB → opérateur lit le JSON → saisie sur le site web	ESP32 → POST HTTP direct (Wi-Fi) → API Flask → site web
Validation manuelle, idéale pour la démonstration en environnement contrôlé	Chaîne entièrement automatisée, exploitable sur le terrain

7. Procédure d'exécution

Pour reproduire le système complet, trois processus parallèles sont à lancer :

Terminal 1 — API Flask (module ML)

```
cd ml_module
python app.py
```

Terminal 2 — Site web React

```
cd website
npm run dev
```

Wokwi — simulation embarquée

Ouvrir le projet dans VS Code, lancer la simulation Wokwi, attendre l'état « STABLE », appuyer sur le bouton, copier le JSON depuis le terminal série et le coller dans la page Tracking pour obtenir la recommandation.

8. Conclusion

L'architecture ESP32 → ML → Web met en œuvre une chaîne complète d'agriculture de précision : capture matérielle des paramètres du sol, inférence par apprentissage automatique, et restitution utilisateur via une interface web moderne. La validation par simulation Wokwi prouve la robustesse du firmware indépendamment du matériel, et la modularité du protocole JSON permet une transition transparente vers un déploiement Wi-Fi autonome.

Chapitre 03 :Partie

data Analytics

1. Introduction :

Ce Chapitre présente une analyse de données agricoles effectuée dans le cadre du module de Data Analysis. L'objectif principal est de construire un modèle de Machine Learning capable de classifier les besoins en nutriments d'un sol à partir de trois indicateurs clés : l'azote (N), le phosphore (P) et le potassium (K).

Le pipeline complet comprend :

- Chargement et exploration du dataset
- Création de la variable cible (Soil_Need)
- Entraînement de deux modèles : Arbre de Décision et Forêt Aléatoire
- Évaluation des performances
- Système de test interactif.

2. Description du Dataset :

Le fichier Agriculture_dataset_with_metadata.csv contient des données collectées par des drones UAV (Unmanned Aerial Vehicle) sur différentes zones agricoles.

```
df = pd.read_csv("Agriculture_dataset_with_metadata.csv")
df.head()
```

Zone ID	Image Source ID	Image Type	NDVI	NDRE	RGB Damage Score	UAV Timestamp	N	P	K	...	NDI Label	PDI Label	Semantic Tag	Action Suggested	Energy Consumed	mAh
Z8	IMG_Z8_202504010600.jpg	Multispectral	0.52	0.52	NaN	4/1/2025 6:00	43	33	94	...	Medium	Medium	P-deficiency, K-deficiency	Apply Pesticide		3.90
Z1	IMG_Z1_202504010605.jpg	Multispectral	0.47	0.12	NaN	4/1/2025 6:05	65	27	39	...	High	High	N-deficiency, Healthy	Monitor		1.18
Z10	IMG_Z10_202504010610.jpg	Multispectral	0.54	0.45	NaN	4/1/2025 6:10	29	40	65	...	Low	Low	P-deficiency, K-deficiency	Apply Fertilizer		2.99
Z5	IMG_Z5_202504010615.jpg	RGB	NaN	NaN	0.31	4/1/2025 6:15	47	37	70	...	Medium	Low	K-deficiency, K-deficiency	Apply Fertilizer		8.86
Z4	IMG_Z4_202504010620.jpg	Multispectral	0.69	0.54	NaN	4/1/2025 6:20	67	28	36	...	Medium	Low	K-deficiency, N-deficiency	Apply Fertilizer		1.19

q × 24 columns

```
print(df.columns)
print(df.shape)
print(df.isnull().sum())
```

```
Index(['Zone_ID', 'Image_Source_ID', 'Image_Type', 'NDVI', 'NDRE',
      'RGB_Damage_Score', 'UAV_Timestamp', 'N', 'P', 'K', 'Moisture', 'pH',
      'Temperature', 'Humidity', 'NDI_Label', 'PDI_Label', 'Semantic_Tag',
      'Action_Suggested', 'Energy_Consumed_mAh', 'Latency_ms', 'Current_Node',
      'Migrated_To', 'Migration_Required', 'Migration_Timestamp'],
      dtype='object')
(6000, 21)
```

3. Préparation des Données :

```
data = df[['N', 'P', 'K']].copy()
data.head()
```

Pytho

	N	P	K
0	43	33	94

Ce choix est justifié par le fait que les besoins en nutriments dépendent directement et uniquement de ces trois indicateurs chimiques du sol.

4. Création de la Variable Cible :

```
def definir_besoin(row):
    besoins = []

    if row['N'] < 40:
        besoins.append("Besoin azote")
    if row['P'] < 30:
        besoins.append("Besoin phosphore")
    if row['K'] < 50:
        besoins.append("Besoin potassium")

    if len(besoins) == 0:
        return "Sol équilibré"

    return " + ".join(besoins)

data['Soil_Need'] = data.apply(definir_besoin, axis=1)
data.head()
```

Pytho

	N	P	K	Soil_Need
0	43	33	94	Sol équilibré
1	65	27	39	Besoin phosphore + Besoin potassium
2	29	40	65	Besoin azote
3	47	37	70	Sol équilibré
4	67	28	36	Besoin phosphore + Besoin potassium

5. Distribution de la Variable Cible :

```
print(data['Soil_Need'].value_counts())
```

Python

```
Soil_Need
Besoin phosphore                13057
Besoin azote + Besoin phosphore    9340
Besoin phosphore + Besoin potassium  8844
Sol équilibré                    7731
Besoin azote + Besoin phosphore + Besoin potassium  6310
Besoin azote                     5747
Besoin potassium                  5186
Besoin azote + Besoin potassium    3785
Name: count, dtype: int64
```

6. Séparation Train / Test :

Le dataset est divisé en 80% pour l'entraînement et 20% pour le test, avec stratification pour préserver la distribution des classes :

```
X = data[['N', 'P', 'K']]
y = data['Soil_Need']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Python

7. Modèles de Machine Learning

❖ **Arbre de Décision (Decision Tree)**

✓ Configuration du modèle

```
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
```

Python

Accuracy : 1.0

	precision	recall	f1-score	support
Besoin azote	1.00	1.00	1.00	1150
Besoin azote + Besoin phosphore	1.00	1.00	1.00	1868
Besoin azote + Besoin phosphore + Besoin potassium	1.00	1.00	1.00	1262
Besoin azote + Besoin potassium	1.00	1.00	1.00	757
Besoin phosphore	1.00	1.00	1.00	2611
Besoin phosphore + Besoin potassium	1.00	1.00	1.00	1769
Besoin potassium	1.00	1.00	1.00	1037
Sol équilibré	1.00	1.00	1.00	1546
accuracy			1.00	12000
macro avg	1.00	1.00	1.00	12000
weighted avg	1.00	1.00	1.00	12000

8. Système de Test Interactif

Un système de test interactif a été intégré pour permettre à un utilisateur de saisir les valeurs N, P, K d'un échantillon de sol et d'obtenir une prédiction en temps réel :


```

print("\n===== TEST DU SOL =====")

while True:
    try:
        N = float(input("Entrer N (azote) : "))
        P = float(input("Entrer P (phosphore) : "))
        K = float(input("Entrer K (potassium) : "))

        sample = pd.DataFrame([[N, P, K]], columns=['N', 'P', 'K'])

        prediction = model.predict(sample)[0]

        print("\n👉 Résultat :", prediction)

    except:
        print("❌ Erreur de saisie")

    choix = input("\nContinuer ? (o/n) : ")
    if choix.lower() != 'o':
        break

```

```
===== TEST DU SOL =====
```

```
👉 Résultat : Besoin azote + Besoin phosphore + Besoin potassium
```

```
===== TEST DU SOL =====
```

```
👉 Résultat : Besoin potassium
```

```
===== TEST DU SOL =====
```

```
👉 Résultat : Sol équilibré
```

```
..
```

```
===== TEST DU SOL =====
```

```
❌ Erreur de saisie
```

9. Conclusion :

Ce chapitre a démontré l'efficacité du Machine Learning et des modèles d'Arbre de Décision et de Forêt Aléatoire dans l'analyse de données agricoles par drones pour optimiser la fertilisation. L'intégration d'un système interactif permet une évaluation rapide et en temps réel des besoins en nutriments du sol.

Chapitre 04 :

L'application

1. Introduction :

Dans ce chapitre, nous présentons notre application mobile qui constitue l'interface principale du projet.

Elle est conçue pour aider les agriculteurs à prendre de meilleures décisions agricoles grâce à la combinaison de l'IoT et de l'analyse de données.

L'application permet de transformer les données techniques issues des capteurs (mesures du sol) en informations simples, claires et utiles, accessibles directement depuis un smartphone.

Elle ne se limite pas uniquement à l'analyse du sol, mais offre également un **système complet de recherche de produits agricoles** afin d'aider l'agriculteur dans ses choix.

2. Objectifs de l'application :

L'application a été développée avec plusieurs objectifs principaux :

- Suivre et analyser l'état du sol en temps réel
- Recommander des produits et cultures adaptés au sol
- Permettre la recherche de n'importe quel produit agricole
- Afficher les prix des produits agricoles
- Fournir la description des produits
- Donner des liens vers des sites de vente fiables
- Aider l'agriculteur à choisir le meilleur produit au meilleur prix
- Offrir une interface simple, rapide et intuitive

3. Principe général de l'application :

L'application est basée sur **deux grandes fonctionnalités complémentaires** :

A. Recherche de produits agricoles :

Dans cette partie, l'agriculteur peut rechercher **n'importe quel type de produit agricole**, par exemple :

- Engrais
- Semences
- Produits phytosanitaires

- Outils agricoles

Pour chaque produit, l'application affiche :

- La photo du produit
- La description du produit
- Les différents prix disponibles
- Des liens vers des sites de vente

L'objectif est de permettre à l'agriculteur de **comparer les offres et choisir le meilleur prix et la meilleure qualité**.

B. Analyse du sol (IoT + Data Analytics) :

a. Partie IoT (Système embarqué) :

Cette partie concerne la collecte des données du sol à l'aide de capteurs :

- PH du sol
- Humidité
- Azote (N)
- Phosphore (P)
- Potassium (K)

Ces capteurs envoient les données en temps réel vers le système.

b. Partie Data Analytics :

Les données collectées sont ensuite analysées pour :

- Comprendre l'état du sol
- Identifier les besoins de la culture
- La prise de décision agricole

Exemples :

- Humidité faible : engrais hydratants + systèmes d'irrigation goutte-à-goutte
- Humidité élevée : produits anti-fongiques + amendements drainants
- PH élevée : semences résistantes à la chaleur + paillage protecteur
- PH basse : engrais de croissance + serres agricoles

c. Résultat dans l'application :

Après analyse, l'application affiche :

- Les produits agricoles adaptés au sol

4. Description des interfaces de l'application :

Dans cette section, nous présentons les différentes interfaces de l'application mobile.

Chaque interface a été conçue pour offrir une expérience simple, intuitive et adaptée aux besoins des agriculteurs.

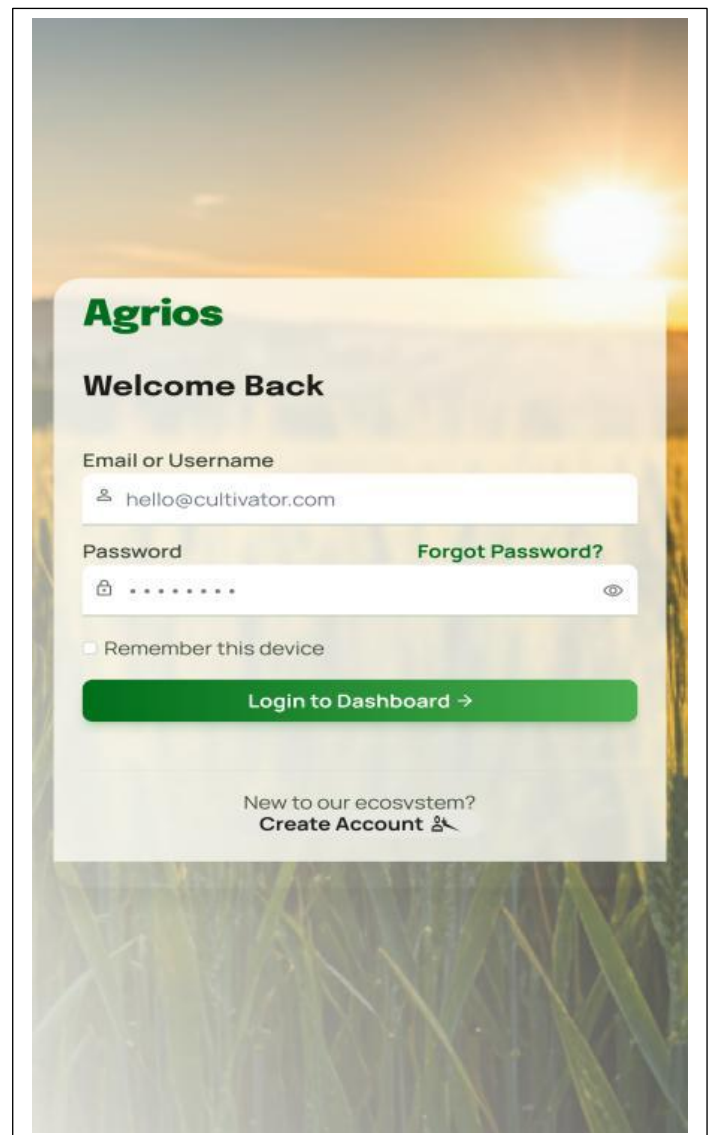
A. Page « Login »:

Cette page permet à l'utilisateur de se connecter à son compte en toute sécurité.

Il doit entrer :

- Son email/nom
- Son mot de passe

Elle garantit l'accès sécurisé à l'application et aux données personnelles.



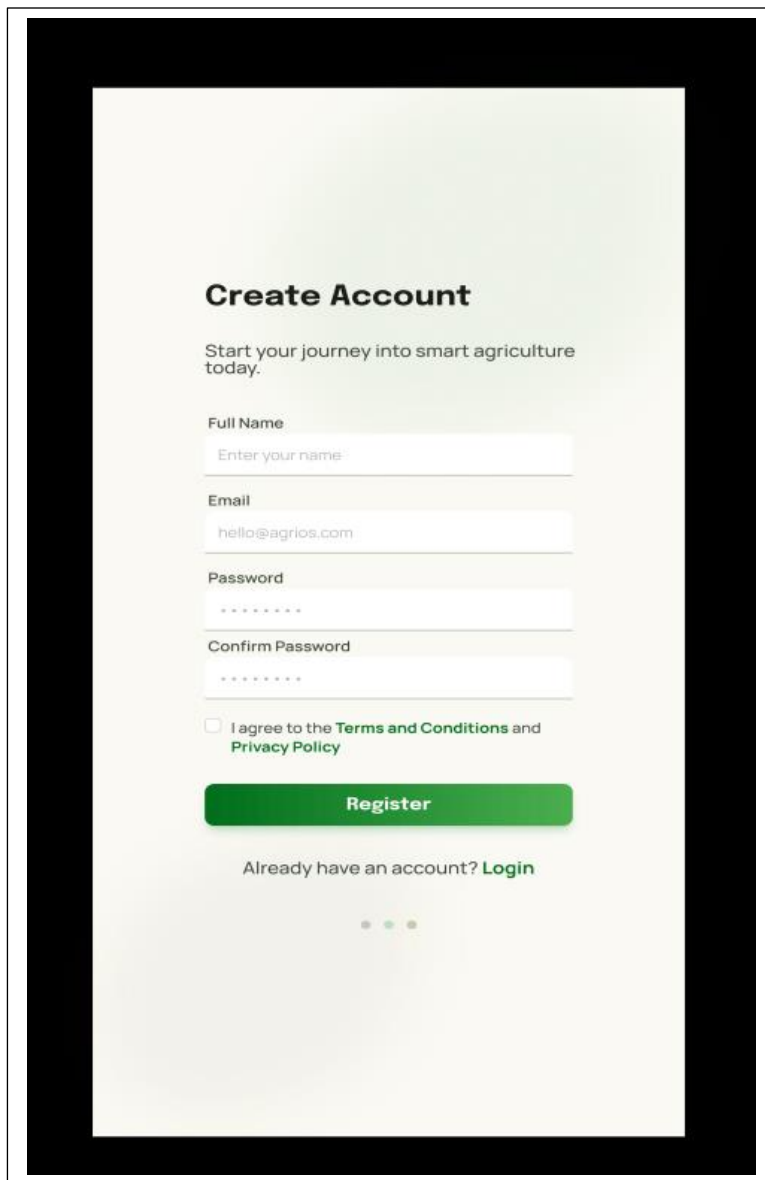
B. Page « Registre » :

Cette page permet aux nouveaux utilisateurs de créer un compte.

Elle contient :

- Nom et prénom
- Email
- Mot de passe
- Confirmer le mot de passe

Elle permet d'enregistrer l'utilisateur dans la base de données pour accéder à l'application.

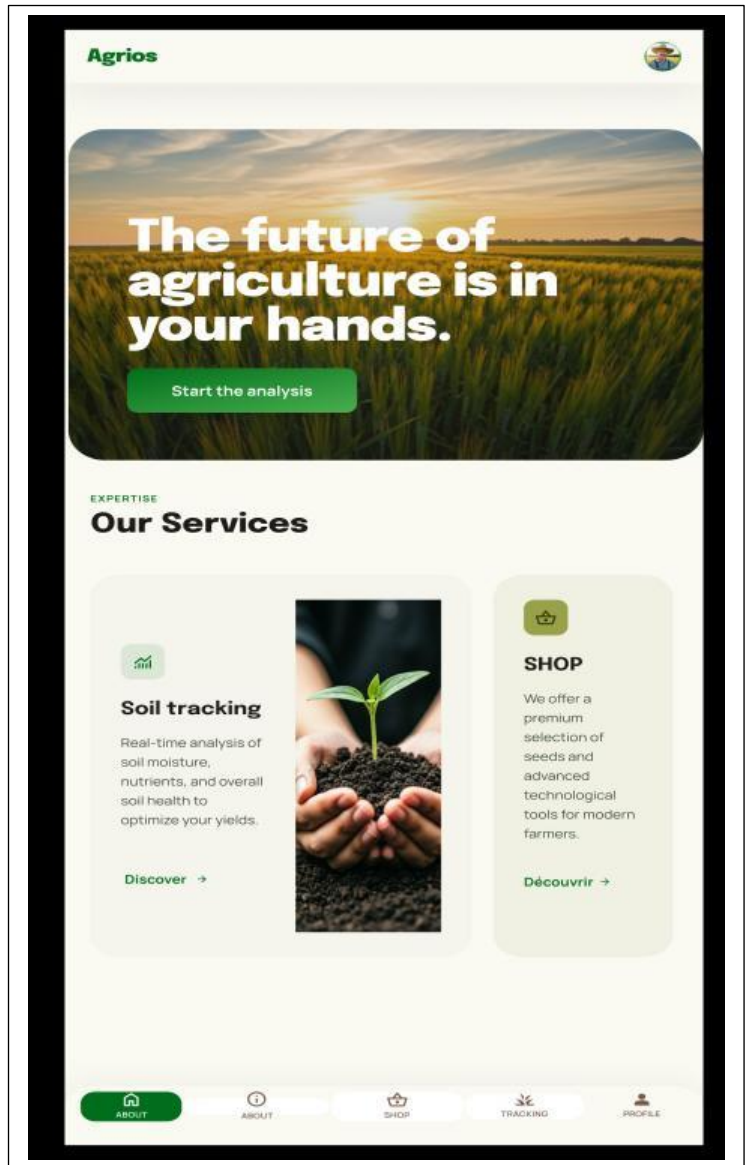
The image shows a mobile application registration screen titled "Create Account". The background is a light beige color with a subtle green leaf pattern. The title "Create Account" is in bold black text. Below it, a subtitle reads "Start your journey into smart agriculture today." The form consists of several input fields: "Full Name" with a placeholder "Enter your name", "Email" with a placeholder "hello@agrios.com", "Password" with a masked input "*****", and "Confirm Password" with a masked input "*****". Below these fields is a checkbox labeled "I agree to the Terms and Conditions and Privacy Policy". A prominent green button with the text "Register" is positioned below the checkbox. At the bottom, there is a link that says "Already have an account? Login". The entire form is enclosed in a white rounded rectangle, which is itself set against a black background.

C. Page «Home » :

C'est la page principale de l'application.

Elle permet :

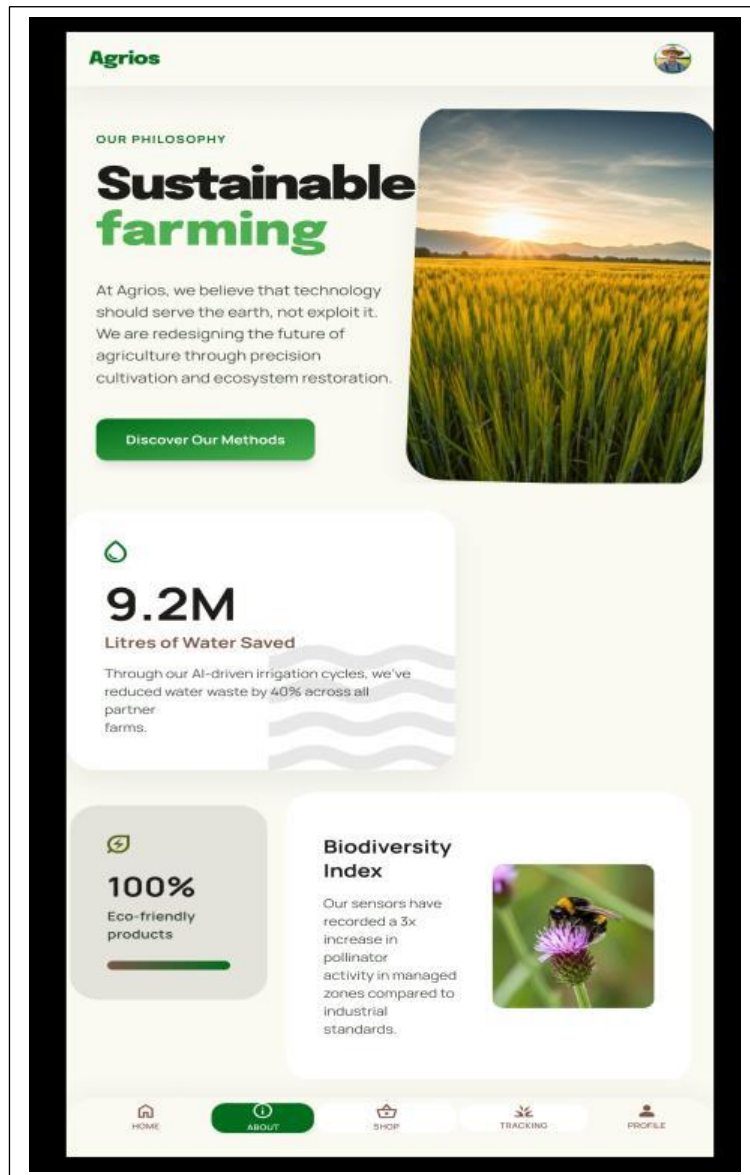
- D'accéder aux différentes fonctionnalités
- De naviguer facilement dans l'application
- D'avoir un aperçu des services proposés
- Elle sert de point central de navigation.



D. Page « About » :

La page About a pour objectif de **présenter la vision et la philosophie du projet Agrios**. Elle explique l'importance de l'agriculture durable et le rôle de la technologie dans la protection de l'environnement.

Cette interface permet à l'utilisateur de comprendre **la mission globale de l'application** et ses valeurs principales.



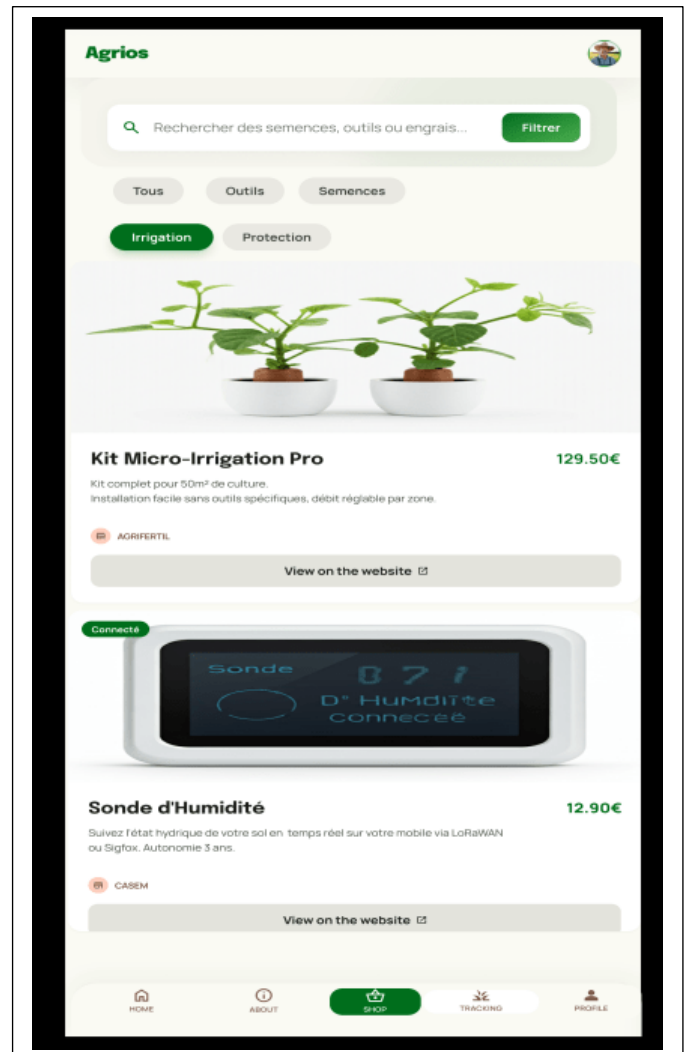
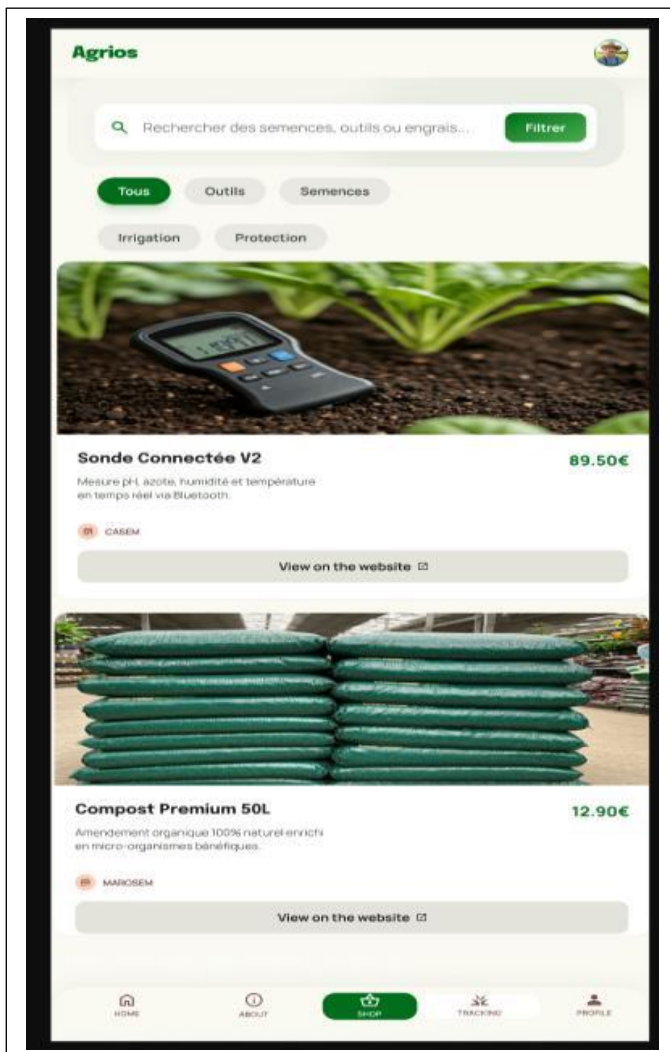
E. Page « Shop » :

Cette page permet à l'utilisateur de rechercher des produits agricoles.

Elle affiche :

- Nom du produit
- Photo du produit
- Description
- Prix
- Liens vers des sites de vente

Elle aide l'agriculteur à comparer et choisir les meilleurs produits.



F. Page «Tracking » :

La page Tracking est directement liée au système IoT et à la partie Data Analytics. Elle permet à l'utilisateur de saisir ou consulter les mesures du sol afin d'obtenir des recommandations adaptées.

L'utilisateur peut entrer les données du sol :

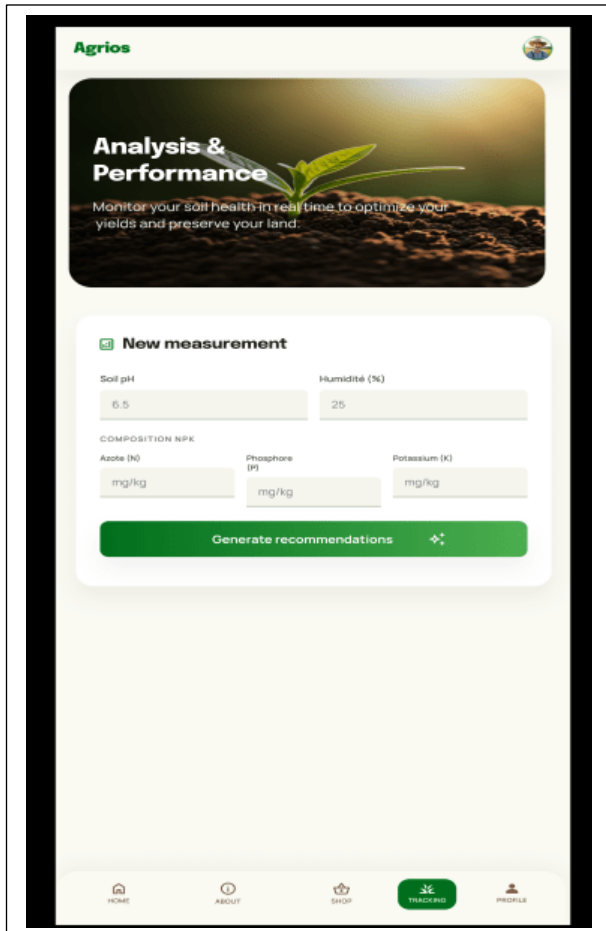
- PH du sol
- Humidité
- Azote (N)
- Phosphore (P)

- Potassium (K)

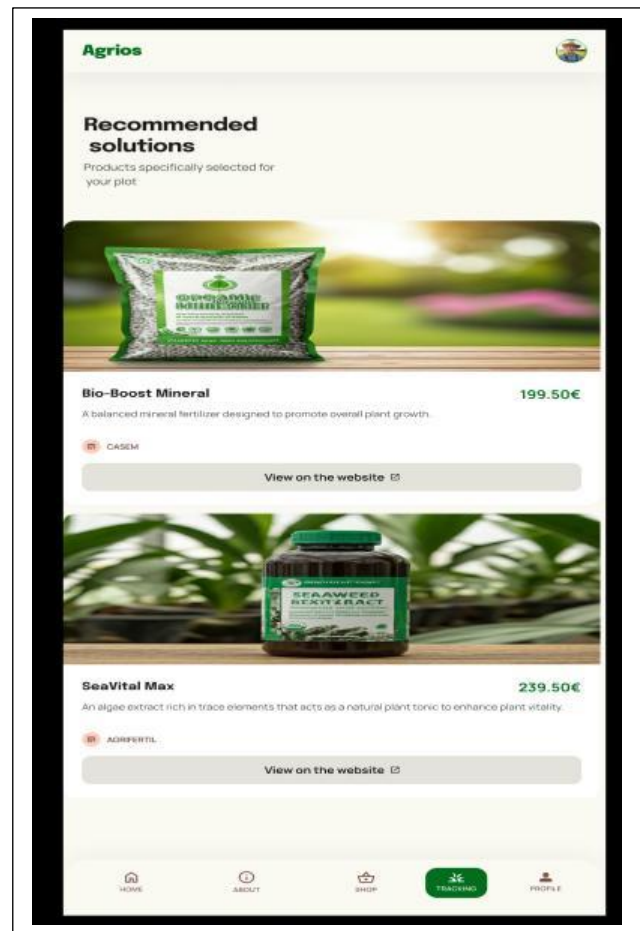
Ces données sont ensuite analysées par le système.

Après analyse, l'application affiche :

- Les produits agricoles recommandés (engrais, semences, etc.)



The screenshot shows the 'Analysis & Performance' section of the Agrios app. It features a header with the app's logo and a sub-header 'Analysis & Performance' with a description: 'Monitor your soil health in real time to optimize your yields and preserve your land.' Below this is a 'New measurement' form. The form has two input fields for 'Soil pH' (value: 6.5) and 'Humidité (%)' (value: 25). Underneath, there's a 'COMPOSITION NPK' section with three input fields: 'Azote (N)' (value: mg/kg), 'Phosphore (P)' (value: mg/kg), and 'Potassium (K)' (value: mg/kg). A green button labeled 'Generate recommendations' with a plus icon is at the bottom of the form. The bottom navigation bar includes icons for HOME, ABOUT, SHOP, TRACKING (highlighted in green), and PROFILE.



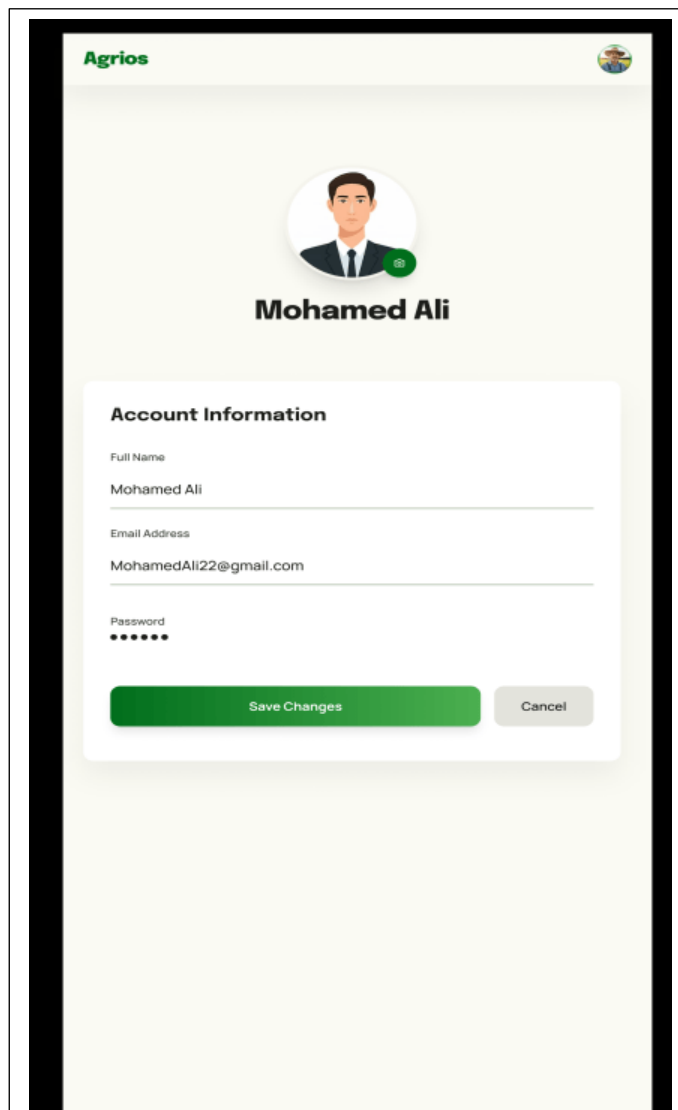
G. Page « Profile » :

La page Profile permet à l'utilisateur de gérer ses informations personnelles au sein de l'application.

L'utilisateur peut modifier :

- Photo de profil
- Nom
- Email
- Mot de passe

Cette page offre un espace personnel sécurisé permettant à l'utilisateur de **mettre à jour ses informations et personnaliser son compte**.



The screenshot displays the 'Agrios' user profile interface. At the top left is the 'Agrios' logo, and at the top right is a small circular icon. Below the header, there is a circular profile picture of a man in a suit, with the name 'Mohamed Ali' centered underneath it. A green edit icon is visible on the bottom right of the profile picture. Below the name, a white card titled 'Account Information' contains the following fields: 'Full Name' with the value 'Mohamed Ali', 'Email Address' with the value 'MohamedAli22@gmail.com', and 'Password' represented by seven dots. At the bottom of the card are two buttons: a green 'Save Changes' button and a grey 'Cancel' button.

5. Conclusion :

Dans ce chapitre, nous avons présenté notre application mobile, qui permet aux agriculteurs de rechercher des produits agricoles et de recevoir des recommandations adaptées à leur sol grâce aux données analysées.

Cette application facilite la prise de décision en combinant simplicité, accessibilité et intelligence des données.

Conclusion Générale

Au terme de ce projet, nous avons développé une solution complète visant à moderniser le secteur agricole grâce à l'utilisation des technologies numériques.

Notre application mobile permet aux agriculteurs de rechercher des produits agricoles, comparer leurs prix et caractéristiques, et accéder à des liens de vente fiables. Elle intègre également un système basé sur l'IoT et le Data Analytics permettant d'analyser l'état du sol et de proposer des solutions adaptées.

Ce projet a permis de mettre en évidence l'importance de la digitalisation dans l'agriculture, en offrant des outils d'aide à la décision simples, efficaces et accessibles.

Ainsi, cette solution contribue à améliorer la productivité, optimiser les ressources et faciliter la prise de décision des agriculteurs, tout en répondant aux défis actuels du secteur.

Enfin, ce travail ouvre des perspectives d'amélioration, notamment l'ajout de nouvelles fonctionnalités, et l'élargissement de la plateforme à un plus grand nombre d'utilisateurs.